```python
import nltk

nltk.download("brown")
nltk.download("universal_tagset")

from nltk.corpus import brown

sentences = brown.sents()

print("Total sentences:", len(sentences))

print("\nFirst sentence:")
print(sentences[0])

print("\nSecond sentence:")
print(sentences[1])
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!

Total sentences: 57340

First sentence:
['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', 'Friday', 'an',
'investigation', 'of', "Atlanta's", 'recent', 'primary', 'election',
'produced', '``', 'no', 'evidence', "''", 'that', 'any',
'irregularities', 'took', 'place', '.']

Second sentence:
['The', 'jury', 'further', 'said', 'in', 'term-end', 'presentments',
'that', 'the', 'City', 'Executive', 'Committee', ',', 'which', 'had',
'over-all', 'charge', 'of', 'the', 'election', ',', '``', 'deserves',
'the', 'praise', 'and', 'thanks', 'of', 'the', 'City', 'of',
'Atlanta', "''", 'for', 'the', 'manner', 'in', 'which', 'the',
'election', 'was', 'conducted', '.']
```

```python
from collections import defaultdict, Counter

sentences = brown.sents()
tokens = [word.lower() for sent in sentences for word in sent]
print("Tokens:", len(tokens))
```

```
Tokens: 1161192
```

```python
def build_ngram(tokens, n):
    model = defaultdict(Counter)
    for i in range(len(tokens)-n):
        context = tuple(tokens[i:i+n-1])
        word = tokens[i+n-1]
```

```
        model[context][word] += 1
    return model

bigram_model = build_ngram(tokens, 2)
trigram_model = build_ngram(tokens, 3)
print("Models Ready")

def predict_next(context, model):
    context = tuple([w.lower() for w in context.split()])
    if context not in model:
        return []
    return [word for word, _ in model[context].most_common(5)]
```

```
Models Ready
```

```
print("Bigram Prediction:", predict_next("speed", bigram_model))
print("Trigram Prediction:", predict_next("that dog", trigram_model))
```

```
Bigram Prediction: [',', 'of', 'and', '.', 'for']
Trigram Prediction: ['.', 'has']
```